

**LAPORAN PRAKTIKUM
PRAKTIKUM PENGUJIAN PERANGKAT LUNAK**

PERTEMUAN 1

JUNIT



Disusun Oleh:

Nama : Abdullah Afif Habiburrohman
NIM : 537611/SV/24441
Kelas : BB / B2
Dosen : Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

**PROGRAM STUDI SARJANA TERAPAN
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

Contents

A	Dasar Teori	2
B	Praktikum	3
B.1	Card.java	5
B.2	Wallet.java	6
B.3	WalletTest.java	7
B.4	GitHub Repository	9
C	Kesimpulan	10

A. Dasar Teori

Dalam pengembangan perangkat lunak berbasis Java, pengelolaan pustaka (library) dan dependensi sering kali menjadi tantangan tersendiri jika dilakukan secara manual. Apache Maven hadir sebagai alat otomatisasi build dan manajemen proyek yang menggunakan konsep Project Object Model (POM). Inti dari konfigurasi Maven terletak pada file pom.xml, di mana pengembang dapat mendefinisikan struktur proyek serta dependensi eksternal yang dibutuhkan.

Pengujian perangkat lunak dilakukan secara bertingkat, dan tingkatan paling dasar dalam hierarki ini adalah unit testing. Fokus utama dari pengujian ini adalah memverifikasi unit terkecil dari perangkat lunak untuk memastikan bahwa unit tersebut menghasilkan keluaran yang benar sesuai dengan masukan yang diberikan. Penerapan unit testing sejak awal pengembangan bertujuan untuk mengidentifikasi kesalahan logika lebih dini sehingga perbaikan dapat dilakukan dengan biaya yang lebih rendah serta meningkatkan kualitas kode secara keseluruhan.

JUnit merupakan salah satu kerangka kerja (framework) pengujian yang paling populer dalam ekosistem Java. JUnit menyediakan mekanisme standar untuk menulis dan menjalankan pengujian otomatis yang dapat diulang (repeatable), sehingga validasi kode dapat dilakukan terus-menerus selama siklus pengembangan.

Berbeda dengan versi pendahulunya yang bersifat monolitik, JUnit 5 (generasi terkini) dibangun dengan arsitektur modular yang terdiri dari tiga sub-proyek utama:

1. JUnit Platform: Berfungsi sebagai fondasi untuk meluncurkan kerangka kerja pengujian di JVM. Ini adalah komponen yang memungkinkan IDE (seperti IntelliJ IDEA) untuk mendeteksi dan menjalankan tes.
2. JUnit Jupiter: Merupakan model pemrograman dan ekstensi baru untuk menulis tes dan ekstensi di JUnit 5. Ini menyediakan anotasi modern seperti `@Test`, `@BeforeEach`, serta metode asersi.
3. JUnit Vintage: Menyediakan mesin uji untuk menjalankan tes yang ditulis menggunakan JUnit 3 dan JUnit 4 demi kompatibilitas ke belakang.

Dalam implementasi praktisnya, sering kali membingungkan untuk menentukan dependensi mana saja yang perlu dimasukkan (apakah hanya API, Engine, atau Params). Oleh karena itu, tersedia artefak bernama JUnit Jupiter (Aggregator).

Inti dari sebuah pengujian unit adalah verifikasi, yang dalam JUnit dilakukan melalui mekanisme Assertion. Asersi mendefinisikan kondisi yang dianggap benar; jika kondisi tersebut terpenuhi, maka tes dianggap lulus (pass), dan jika tidak, tes dianggap gagal (fail).

JUnit menyediakan berbagai metode statis untuk keperluan ini, antara lain:

1. `assertEquals`: Membandingkan kesesuaian antara nilai yang diharapkan (expected) dengan nilai aktual yang dihasilkan kode.
2. `assertTrue`: Memastikan sebuah kondisi bernilai benar.

3. `assertNotNull` dan `assertNull`: Memeriksa keberadaan objek (apakah null atau tidak).

Selain itu, untuk menangani skenario pengujian yang kompleks, JUnit Jupiter menyediakan metode `assertAll`. Metode ini memungkinkan eksekusi beberapa asersi sekaligus dalam satu blok pengujian. Keunggulannya adalah jika asersi pertama gagal, asersi berikutnya tetap akan dieksekusi, sehingga pengembang bisa mendapatkan laporan kesalahan yang lengkap dalam satu kali jalan.

B. Praktikum

Mahasiswa membuat proyek Maven baru dengan nama `Pt1` menggunakan IntelliJ IDEA.

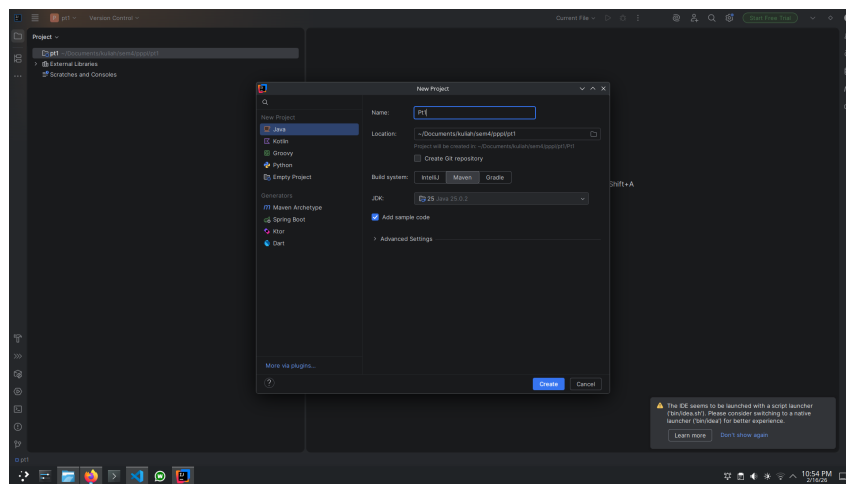


Figure 1: IntelliJ IDEA - Proyek Maven Pt1

Kemudian, mahasiswa menambahkan dependensi JUnit Jupiter (Aggregator) ke dalam file `pom.xml` untuk memastikan semua komponen yang diperlukan untuk pengujian unit tersedia.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5         ↪ http://maven.apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7     <groupId>org.example</groupId>
8     <artifactId>Pt1</artifactId>
9     <version>1.0-SNAPSHOT</version>
10
11     <properties>
12         <maven.compiler.source>25</maven.compiler.source>
13         <maven.compiler.target>25</maven.compiler.target>
14         <project.build.sourceEncoding>UTF-8</project.build.
15             ↪ sourceEncoding>
```

```

15     </properties>
16
17     <dependencies>
18         <!-- Source: https://mvnrepository.com/artifact/org.
           ↳ junit.jupiter/junit-jupiter -->
19         <dependency>
20             <groupId>org.junit.jupiter</groupId>
21             <artifactId>junit-jupiter</artifactId>
22             <version>6.0.1</version>
23             <scope>test</scope>
24         </dependency>
25     </dependencies>
26
27 </project>

```

Setelah menambahkan dependensi, mahasiswa melakukan sinkronisasi proyek untuk memastikan bahwa semua library yang diperlukan diunduh dan tersedia untuk pengembangan serta pengujian.

Mahasiswa diminta untuk mengerjakan tugas:

2.4 Tugas Sebuah kelas Wallet memiliki :

1. atribut owner
2. atribut berupa list kartu-kartu
3. atribut untuk menyimpan list uang lembaran dan koin
4. Fungsionalitas untuk :

Set data owner, menambahkan kartu, mengambil kartu, menambahkan uang rupiah, mengambil uang, menampilkan jumlah uang yang ada di dompet. Implementasikan kelas tersebut dan buatlah unit testing yang sesuai dengan fungsionalitas di atas. Manfaatkanlah berbagai macam assertion yang sudah dipelajari. Method yang diimplementasikan juga bisa ditambahkan sesuai dengan kebutuhan.

Jawaban:

Struktur direktori proyek Maven yang dibuat untuk tugas ini adalah sebagai berikut:

Pt1 tree

```

.
|-- pom.xml
|-- src
|   |-- main
|   |   |-- java
|   |   |   |-- org
|   |   |   |-- example

```

```
| | | |-- Card.java
| | | '-- Wallet.java
| | '-- resources
| '-- test
|   '-- java
|     '-- org
|       '-- example
|         '-- WalletTest.java
'-- target
    |-- classes
    |   '-- org
    |     '-- example
    |       |-- Card.class
    |       |-- Main.class
    |       '-- Wallet.class
    |-- generated-sources
    |   '-- annotations
    |-- generated-test-sources
    |   '-- test-annotations
    '-- test-classes
        '-- org
          '-- example
            '-- WalletTest.class
```

22 directories, 8 files

Terdapat 3 file utama yang digunakan untuk tugas ini, yaitu:

B.1 Card.java

```
1 package org.example;
2
3 public class Card {
4     private String namaBank;
5     private String nomorRekening;
6
7     public Card(String namaBank, String nomorRekening) {
8         this.namaBank = namaBank;
9         this.nomorRekening = nomorRekening;
10    }
11
12    public String getNamaBank() {
13        return namaBank;
14    }
15
16    public String getNomorRekening() {
17        return nomorRekening;
18    }
19 }
```

1. `package org.example;` untuk mendefinisikan namespace dari kelas `Card`, yang berada dalam paket `org.example`.
2. Deklarasi kelas `Card` dengan dua atribut `private`: `namaBank` (nama bank penerbit kartu) dan `nomorRekening` (nomor rekening yang terkait dengan kartu).
3. Konstruktor `Card` untuk menginisialisasi atribut `namaBank` dan `nomorRekening` saat objek `Card` dibuat.
4. Getter `getNamaBank()` untuk mengakses nilai `namaBank`.
5. Getter `getNomorRekening()` untuk mengakses nilai `nomorRekening`.

B.2 Wallet.java

```
1 package org.example;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class Wallet {
7     private String owner;
8     private List<Card> cards;
9     private List<Integer> moneys;
10
11     public Wallet(String owner) {
12         this.owner = owner;
13         this.cards = new ArrayList<>();
14         this.moneys = new ArrayList<>();
15     }
16
17     public void setOwner(String owner) {
18         this.owner = owner;
19     }
20
21     public String getOwner() {
22         return owner;
23     }
24
25     public void addCard(String bank, String nomor) {
26         Card newCard = new Card(bank, nomor);
27         this.cards.add(newCard);
28     }
29
30     public Card takeCard(String nomor) {
31         for (Card card : cards) {
32             if (card.getNomorRekening().equals(nomor)) {
33                 cards.remove(card);
34                 return card;
35             }
36         }
37         return null;
38     }
39 }
```

```

40     public List<Card> getCards() {
41         return cards;
42     }
43
44     public void addMoney(Integer lembaran) {
45         if (lembaran > 0) {
46             this.moneys.add(lembaran);
47         }
48     }
49
50     public boolean takeMoney(Integer lembaran) {
51         if (this.moneys.contains(lembaran)) {
52             this.moneys.remove(lembaran);
53             return true;
54         }
55         return false;
56     }
57
58     public int calculateTotalBalance() {
59         int total = 0;
60         for (Integer uang : this.moneys) {
61             total += uang;
62         }
63         return total;
64     }
65
66     public List<Integer> getMoneys() {
67         return moneys;
68     }
69 }

```

1. 'package' dan 'import': Menentukan paket 'org.example' dan mengimpor 'ArrayList'/'List' untuk struktur data.
2. Atribut kelas: 'owner' (pemilik), 'cards' (list objek 'Card'), dan 'moneys' (list pecahan uang sebagai 'Integer').
3. Konstruktor: Menginisialisasi 'owner' serta membuat instance kosong untuk 'cards' dan 'moneys'.
4. Operasi kartu: 'addCard(...)' menambah 'Card'; 'takeCard(...)' mencari berdasarkan nomor, menghapus dan mengembalikan 'Card' (atau 'null'). 'getCards()' mengembalikan daftar kartu.
5. Operasi uang: 'addMoney(...)' menambahkan lembaran positif; 'takeMoney(...)' menghapus pecahan jika ada dan mengembalikan 'boolean' keberhasilan; 'calculateTotalBalance()' menjumlahkan semua pecahan.
6. Getter tambahan: 'getOwner()'/'setOwner()' dan 'getMoneys()' untuk akses data.

B.3 WalletTest.java

```

1 package org.example;

```

```

2
3 import org.junit.jupiter.api.Test;
4 import org.junit.jupiter.api.DisplayName;
5 import static org.junit.jupiter.api.Assertions.*;
6
7 class WalletTest {
8
9     @Test
10    @DisplayName("Test Inisialisasi Wallet dan Owner")
11    void testWalletOwner() {
12        Wallet dompet = new Wallet("Budi");
13        assertNotNull(dompet, "Objek dompet tidak boleh null");
14        assertEquals("Budi", dompet.getOwner());
15        dompet.setOwner("Andi");
16        assertEquals("Andi", dompet.getOwner());
17    }
18
19    @Test
20    @DisplayName("Test Menambah dan Mengambil Kartu")
21    void testCardOperations() {
22        Wallet dompet = new Wallet("Siti");
23
24        dompet.addCard("BCA", "111-222");
25        dompet.addCard("Mandiri", "333-444");
26        assertEquals(2, dompet.getCards().size());
27        Card diambil = dompet.takeCard("111-222");
28
29        assertAll("Verifikasi Kartu",
30            () -> assertNotNull(diambil),
31            () -> assertEquals("BCA", diambil.getNamaBank())
32            ↪ ),
33            () -> assertEquals("111-222", diambil.
34            ↪ getNomorRekening())
35            );
36
37        assertEquals(1, dompet.getCards().size());
38    }
39
40    @Test
41    @DisplayName("Test Tambah Uang dan Hitung Total")
42    void testAddMoney() {
43        Wallet dompet = new Wallet("Joko");
44        assertEquals(0, dompet.calculateTotalBalance());
45
46        dompet.addMoney(10000);
47        dompet.addMoney(5000);
48        dompet.addMoney(500);
49
50        assertEquals(15500, dompet.calculateTotalBalance());
51    }
52
53    @Test
54    @DisplayName("Test Mengambil Uang (Withdraw)")
55    void testTakeMoney() {
56        Wallet dompet = new Wallet("Lina");
57        dompet.addMoney(50000);

```

```

56         dompet.addMoney(20000);
57
58         boolean isSuccess = dompet.takeMoney(20000);
59
60         assertTrue(isSuccess, "Harusnya berhasil mengambil uang
→ yang tersedia");
61         assertEquals(50000, dompet.calculateTotalBalance(), "
→ Sisa saldo harus 50.000");
62
63         boolean isFail = dompet.takeMoney(100000);
64
65         assertFalse(isFail, "Harusnya gagal mengambil uang yang
→ tidak ada");
66         assertEquals(50000, dompet.calculateTotalBalance(), "
→ Saldo tidak boleh berubah jika gagal");
67     }
68 }

```

1. 'package' dan 'import' JUnit 5 ('@Test', '@DisplayName', dan 'Assertions').
2. Deklarasi kelas 'WalletTest' yang mengelompokkan semua kasus uji untuk 'Wallet'.
3. 'testWalletOwner()': Memverifikasi inisialisasi objek dan 'get/setOwner()' menggunakan 'assertNotNull' dan 'assertEquals'.
4. 'testCardOperations()': Menguji 'addCard()' dan 'takeCard()'; menggunakan 'assertEquals' untuk ukuran daftar dan 'assertAll' untuk mengelompokkan beberapa pemeriksaan pada objek 'Card'.
5. 'testAddMoney()': Mengetes 'addMoney()' dan 'calculateTotalBalance()' (saldo awal 0 dan penjumlahan setelah penambahan).
6. 'testTakeMoney()': Memvalidasi pengambilan uang yang berhasil dan gagal ('assertTrue'/'assertFalse') serta memastikan saldo konsisten.
7. Catatan: Setiap test diberi '@DisplayName' untuk keterbacaan hasil tes; kombinasi 'assertAll' dan assertions individual memberikan laporan kegagalan yang informatif.

Kode dites dan menghasilkan output yang menunjukkan semua tes berhasil (green bar) tanpa kegagalan, menandakan bahwa implementasi 'Wallet' telah memenuhi semua skenario pengujian yang dirancang.

B.4 GitHub Repository

Kode sumber praktikum ini dapat diakses di: <https://github.com/bukunya/PPPL/tree/Pertemuan1>

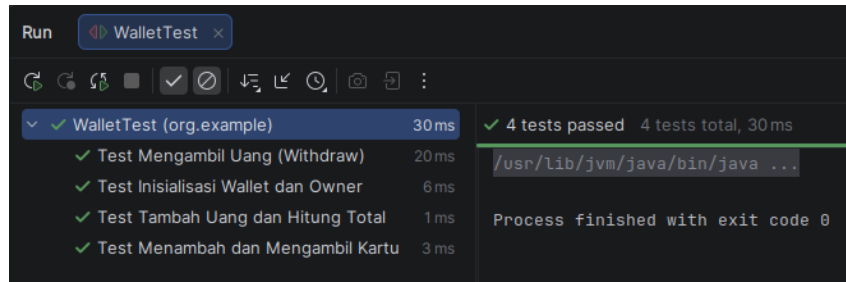


Figure 2: Hasil tes JUnit menunjukkan sukses

C. Kesimpulan

Berdasarkan teori, implementasi, dan pengujian yang dilakukan pada praktikum ini dapat disimpulkan bahwa JUnit 6 menyediakan kerangka kerja pengujian unit yang modular dan fleksibel (Platform, Jupiter). Penggunaan assertion seperti `'assertEquals'`, `'assertTrue'`, `'assertNotNull'`/`'assertNull'` serta konstruk `'assertAll'` memudahkan verifikasi perilaku unit dan memberikan pelaporan kegagalan yang informatif. Implementasi kelas `'Wallet'` memenuhi fungsionalitas yang diminta (manajemen owner, kartu, penambahan/pengambilan uang, perhitungan saldo) dan divalidasi melalui serangkaian unit test di `'WalletTest'`.

Hasil pengujian menunjukkan kasus-kasus uji yang dirancang berjalan sukses, yang meningkatkan keandalan dan mempermudah perawatan kode. Secara keseluruhan, praktikum ini menegaskan pentingnya pengujian unit dalam menjaga kualitas perangkat lunak dan memudahkan proses pengembangan lanjutan.